

---

# 200 Python Interview Questions & answers

Advanced Topics & Core Concepts

**Q001 What is the walrus operator (:=)?**

The walrus operator (:=), introduced in Python 3.8 (PEP 572), is the assignment expression operator. It assigns a value to a variable as part of a larger expression. Example: `while chunk := f.read(8192): process(chunk)`. It avoids redundant computation by assigning and testing in the same expression.

**Q002 What is the 'dataclasses' module?**

The dataclasses module (Python 3.7+) provides a `@dataclass` decorator that auto-generates `__init__`, `__repr__`, and `__eq__` methods for classes based on annotated fields. It reduces boilerplate for data-holding classes. Optional parameters include `frozen=True` (immutable), `order=True` (comparison methods), and `field()` for default values.

**Q003 What are Python type hints?**

Type hints (introduced in Python 3.5 via PEP 484) allow you to annotate variables and function signatures with expected types. Example: `def greet(name: str) -> str`. They are not enforced at runtime but enable static analysis with tools like mypy, improve IDE support, and make code more self-documenting.

**Q004 What is async/await in Python?**

`async/await` is Python's syntax for writing asynchronous, non-blocking code using coroutines. `'async def'` defines a coroutine function. `'await'` suspends the coroutine until the awaited task completes, yielding control to the event loop. Used with `asyncio` for efficient I/O-bound concurrency without threads.

**Q005 What is the 'asyncio' module?**

`asyncio` is Python's standard library for writing concurrent I/O-bound code using an event loop and coroutines. Key functions: `asyncio.run()`, `asyncio.gather()` (run coroutines concurrently), `asyncio.sleep()`, and `asyncio.create_task()`. It is the foundation for async frameworks like FastAPI and aiohttp.

**Q006 What is the 'multiprocessing' module?**

The multiprocessing module enables true parallelism by spawning separate processes, each with its own Python interpreter and memory space — bypassing the GIL. Ideal for CPU-bound tasks. Key classes: Process, Pool (for parallel map/apply), Queue, and Pipe for inter-process communication.

**Q007 What is the 'threading' module?**

The threading module enables concurrent execution using threads within the same process. Threads share memory, requiring synchronization (Lock, RLock, Semaphore, Event) to avoid race conditions. Due to the GIL, threads are best suited for I/O-bound tasks, not CPU-bound parallel computation.

**Q008 What is memoization and how is it implemented in Python?**

Memoization is an optimization technique that caches the results of expensive function calls so they are not recomputed for the same inputs. In Python, use `@functools.lru_cache(maxsize=None)` or `@functools.cache` (Python 3.9+) as a decorator above a recursive or expensive function.

**Q009 What is the 'functools' module?**

The functools module provides higher-order functions. Key tools: `functools.reduce()` (applies a function cumulatively), `@functools.lru_cache` (memoization), `functools.partial()` (partially applies a function with preset arguments), and `functools.wraps()` (preserves metadata of wrapped functions in decorators).

**Q010 What is the 'itertools' module?**

The itertools module provides fast, memory-efficient tools for iterators. Key functions: `chain()` (chains iterables), `cycle()` (infinitely cycles), `combinations()` and `permutations()` (combinatorics), `product()` (cartesian product), `groupby()` (groups consecutive elements), and `islice()` (slices an iterator without materializing it).

**Q011 What is a namedtuple?**

A namedtuple is a factory function from the collections module that creates tuple subclasses with named fields. Example: `Point = namedtuple('Point', ['x', 'y'])`; `p = Point(3, 4)`; `p.x` gives 3. Namedtuples are immutable, memory-efficient, and more readable than plain tuples.

**Q012 What is the 'collections' module?**

The collections module provides specialized container data types: Counter (counts hashable objects), defaultdict (dict with a default factory for missing keys), OrderedDict (remembers insertion order), deque (double-ended queue with O(1) append/pop from both ends), and namedtuple (tuple with named fields).

**Q013 What is the 'heapq' module?**

The heapq module implements a min-heap (priority queue) using a regular Python list. Key functions: `heapq.heappush()`, `heapq.heappop()` (removes smallest), `heapq.heapify()` (converts list in-place), `heapq.nlargest()`, and `heapq.nsmallest()`. Useful for efficiently finding top-K elements.

**Q014 What is multiple inheritance and MRO?**

Multiple inheritance allows a class to inherit from more than one parent. The Method Resolution Order (MRO) defines the search order when looking up a method. Python uses the C3 linearization algorithm. Inspect it using `ClassName.__mro__` or `ClassName.mro()`. `super()` follows the MRO automatically.

**Q015 What is a metaclass in Python?**

A metaclass is a class whose instances are classes themselves. It controls how a class is created. The default metaclass is 'type'. You define a custom metaclass by inheriting from 'type' and overriding `__new__` or `__init__`. Used in frameworks like Django ORM for declarative class definitions.

**Q016 What is `__new__` and how does it differ from `__init__`?**

`__new__` creates and returns a new instance of a class — called before `__init__`. `__init__` then initializes the already-created instance. `__new__` is useful when subclassing immutable types like `int` or `str`, or when implementing singleton patterns where you control instance creation directly.

**Q017 What is the difference between `__getattr__` and `__getattribute__`?**

`__getattribute__` is called on every attribute access. `__getattr__` is only called when the attribute was not found through the normal mechanism — it is a fallback. Overriding `__getattribute__` carelessly can cause infinite recursion; always call `super().__getattribute__()` inside it.

**Q018 What are property decorators?**

The `@property` decorator allows a method to be accessed like an attribute. It enables getter, setter, and deleter functionality with `@property`, `@name.setter`, and `@name.deleter`. This enforces encapsulation by validating or computing values on access without exposing internal implementation details.

**Q019 What is a descriptor in Python?**

A descriptor is any object that defines `__get__`, `__set__`, or `__delete__` methods, controlling attribute access on another class. They are the mechanism behind `@property`, `@classmethod`, and `@staticmethod`. Data descriptors define both `__get__` and `__set__`; non-data descriptors define only `__get__`.

**Q020 What is the purpose of `__slots__`?**

`__slots__` restricts instance attributes to a fixed set, replacing the default per-instance `__dict__`. This reduces memory usage significantly for classes with many instances and speeds up attribute access. Instances with `__slots__` cannot have arbitrary new attributes added at runtime.

**Q021 What is operator overloading in Python?**

Operator overloading allows custom classes to define behavior for built-in operators using dunder methods. Examples: `__add__` for `+`, `__sub__` for `-`, `__mul__` for `*`, `__lt__` for `<`, `__eq__` for `==`, `__len__` for `len()`, `__contains__` for `'in'`. This allows objects to integrate naturally with Python syntax.

**Q022 What is the difference between 'global' and 'nonlocal'?**

'global x' inside a function declares that 'x' refers to the module-level global variable. 'nonlocal x' inside a nested function refers to the variable in the nearest enclosing (non-global) scope. Neither keyword is needed just to read a variable — only to reassign it.

**Q023 What is exception chaining in Python?**

Exception chaining links exceptions using 'raise NewException from original'. The 'from' clause sets the `__cause__` attribute. When an exception occurs inside an except block without 'from', Python sets `__context__` implicitly. Use 'raise X from None' to suppress the original exception context entirely.

**Q024 What is the difference between 'raise' and 'raise from'?**

'raise ExceptionType(message)' raises a new exception. Bare 'raise' re-raises the current exception inside an except block. 'raise NewException from original' explicitly chains exceptions, setting `__cause__` on the new exception. Use 'raise X from None' to hide the original exception entirely.

**Q025 What is string interning in Python?**

String interning is an optimization where Python reuses the same object for identical string literals, saving memory. Short strings and identifiers are typically interned automatically. You can manually intern strings using `sys.intern()`. Never rely on 'is' for string equality checks — always use '=='.

**Q026 What are Python's bitwise operators?**

Python's bitwise operators work on integers at the binary level: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift). Example: 5 & 3 = 1 (0101 & 0011 = 0001). Used in low-level programming, flags, masking, and performance-critical bit manipulation.

**Q027 What is the 'contextlib' module?**

The contextlib module provides utilities for context managers. Key tools: @contextlib.contextmanager (turns a generator into a context manager), contextlib.suppress(exception) (suppresses specified exceptions), contextlib.nullcontext() (a no-op context manager), and ExitStack (dynamic stacking of context managers).

**Q028 What is the difference between 'bytes', 'bytearray', and 'memoryview'?**

bytes is an immutable sequence of integers (0–255) for binary data. bytearray is its mutable counterpart. memoryview exposes the buffer protocol of an object, allowing slicing and working with binary data without copying it — critical for performance when working with large binary buffers.

**Q029 What is the 'abc' module?**

The abc (Abstract Base Classes) module provides infrastructure for defining abstract base classes. Using @abc.abstractmethod, you declare methods subclasses must implement. A class with unimplemented abstract methods cannot be instantiated — raises TypeError. ABCs enforce interface contracts throughout a class hierarchy.

**Q030 What is the 'struct' module?**

The struct module converts between Python values and C structs represented as bytes. Used for binary data parsing and packing — reading binary file formats or network protocols. struct.pack(format, values) returns bytes; struct.unpack(format, buffer) returns a tuple. Format strings use 'i' (int), 'f' (float), 's' (char[]).

**Q031 What is the 'os' module used for?**

The os module provides a portable interface to operating system functionality: os.path for path manipulation (join, exists, dirname), os.listdir() to list directory contents, os.getcwd() for current directory, os.makedirs() to create directories, os.environ for environment variables, and os.remove() to delete files.

**Q032 What is the 'sys' module used for?**

The sys module provides access to interpreter-specific variables and functions: sys.argv (command-line arguments), sys.path (module search path), sys.exit() (exit the interpreter), sys.stdin/stdout/stderr (standard I/O streams), sys.version (Python version string), and sys.getrecursionlimit().

**Q033 What is the difference between Python 2 and Python 3?**

Python 3 introduced print as a function, true division by default, Unicode strings by default, and removed xrange(). Python 2 reached end-of-life in January 2020. Python 3 also introduced f-strings, type hints, async/await, and many standard library improvements not available in Python 2.

**Q034 What is the difference between any() and all()?**

any(iterable) returns True if at least one element is truthy; False if the iterable is empty. all(iterable) returns True if all elements are truthy; also True if the iterable is empty. Both use short-circuit evaluation — they stop processing as soon as the result is determined.

**Q035 What is the zip() function?**

zip() takes multiple iterables and returns an iterator of tuples, pairing elements at the same index. Example: zip([1,2,3], ['a','b','c']) gives (1,'a'), (2,'b'), (3,'c'). It stops at the shortest iterable. Use itertools.zip\_longest() to fill missing values with a fillvalue.



**Q036 What is the enumerate() function?**

`enumerate(iterable, start=0)` returns an iterator of (index, value) tuples, allowing you to loop with both the index and value simultaneously. Example: `for i, val in enumerate(['a','b','c'])` gives (0,'a'), (1,'b'), (2,'c'). The 'start' parameter sets the initial counter value.

**Q037 What is the re module in Python?**

The `re` module provides regular expression support. Key functions: `re.match()` (matches at start), `re.search()` (searches anywhere), `re.findall()` (returns all matches as a list), `re.sub()` (replaces matches), `re.compile()` (compiles a pattern for reuse). Use raw strings (`r'\d+'`) to avoid backslash conflicts.

**Q038 What is the difference between re.match() and re.search()?**

`re.match()` only matches at the beginning of the string — if the pattern is not at the start, it returns `None`. `re.search()` scans through the entire string for any location where the pattern matches. To match the full string, use `re.fullmatch()` or add `^` and `$` anchors.

**Q039 What is JSON and how does Python work with it?**

JSON is a lightweight data-interchange format. Python's `json` module handles serialization and deserialization: `json.dumps(obj)` converts a Python object to a JSON string; `json.loads(string)` parses JSON to a Python object. `json.dump()` and `json.load()` work with file objects directly.

**Q040 What is Python's logging best practice?**

Use the logging module instead of `print()` for production code. Create a logger with `logging.getLogger(__name__)`. Use levels: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`. Configure handlers and formatters to control output. Use lazy formatting: `logger.debug('%s', value)` instead of `logger.debug(f'{value}')`.

**Q041 What is chained comparison in Python?**

Python allows chaining comparisons:  $1 < x < 10$  is evaluated as  $(1 < x)$  and  $(x < 10)$ , where  $x$  is evaluated only once. This is more readable and efficient than writing them separately. Chaining works with all comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`.

**Q042 What is the difference between list, tuple, and array?**

list is a dynamic, mutable sequence holding any type. tuple is immutable and slightly more memory-efficient. `array.array` (from the `array` module) stores elements of a single C-type, making it much more memory-efficient for large numeric data. For numerical computing, NumPy arrays are preferred over all three.

**Q043 What is the difference between `os.path.join` and string concatenation?**

`os.path.join()` correctly constructs file paths by inserting the OS-appropriate separator (`/` on Unix, `\` on Windows) and handling edge cases like trailing slashes. String concatenation is error-prone and platform-dependent. Python 3.4+ `pathlib.Path` provides an even more readable object-oriented approach.

**Q044 What is the difference between `str.format()` and f-strings?**

`str.format()` uses `{}` placeholders: `'{} {}'.format('hello', 'world')`. F-strings (Python 3.6+) embed expressions directly: `f'{name.upper()}'`. F-strings are faster at runtime and more readable. For Python <3.6, `format()` is the preferred modern alternative to %-style formatting.

**Q045 What is Python's numeric precision issue with floats?**

Floating-point numbers follow IEEE 754 standard, which can cause precision issues like  $0.1 + 0.2 \neq 0.3$ , because many decimal fractions cannot be represented exactly in binary. For precise decimal arithmetic, use the `decimal.Decimal` class. Always use `Decimal` instead of `float` for financial calculations.

**Q046 What is a Python virtual environment?**

A virtual environment is an isolated Python environment with its own interpreter and packages, separate from the system Python. It prevents dependency conflicts between projects. Created using 'python -m venv env\_name', activated with 'source env\_name/bin/activate' (Linux/Mac) or 'env\_name\Scripts\activate' (Windows).

**Q047 What are Python's string methods split(), join(), strip()?**

split(sep) splits a string into a list by separator (default: whitespace). join(iterable) joins strings with the calling string as separator: ', '.join(['a','b','c']) gives 'a, b, c'. strip() removes leading and trailing whitespace (or specified characters); lstrip() and rstrip() strip from one side only.

**Q048 What is the difference between input() and raw\_input()?**

In Python 3, input() reads a line from stdin and always returns a string. In Python 2, raw\_input() did the same, while input() evaluated the expression (a security risk). Python 3 removed raw\_input() entirely and made input() safe by always returning a string that you explicitly convert.

**Q049 What is pickling and unpickling?**

Pickling serializes a Python object into a byte stream using the pickle module, so it can be saved to a file or transmitted. Unpickling is the reverse — deserializing the byte stream back into a Python object. Use pickle.dump() to pickle and pickle.load() to unpickle. Never unpickle data from untrusted sources.

**Q050 What is duck typing?**

Duck typing is a concept where the type of an object is less important than the methods or attributes it has. If an object behaves like a duck (has the needed methods), Python treats it as one. This allows for flexible, dynamic code without strict type checking — enabling polymorphism without inheritance.

**Q051 What is the difference between a shallow copy and a deep copy?**

A shallow copy (`copy.copy()`) creates a new container but shares references to nested objects. A deep copy (`copy.deepcopy()`) creates a fully independent object with all nested objects also copied recursively. Assignment (`b = a`) makes both variables point to the same object — no copy at all.

**Q052 What is the difference between positional and keyword arguments?**

Positional arguments are passed in order and matched by position. Keyword arguments are passed by explicitly naming the parameter (`key=value`), allowing them to be in any order. Keyword arguments improve readability and allow skipping optional parameters that have default values.

**Q053 What is the difference between 'assert' and raising an exception?**

'assert condition, message' raises `AssertionError` if False. It can be disabled with the `-O` flag and should never be used for production validation. Raising exceptions (`raise ValueError(...)`) is the correct approach for runtime error handling that must not be silently bypassed in optimized builds.

**Q054 What is the difference between @staticmethod and @classmethod?**

`@staticmethod` defines a method with no implicit first argument — a plain function namespaced inside a class. `@classmethod` receives 'cls' (the class itself) as the first argument and can access or modify class-level state. `@classmethod` is commonly used as an alternative constructor pattern.

**Q055 What is the difference between mutable and immutable objects?**

Mutable objects can be changed after creation — lists, dictionaries, and sets. Immutable objects cannot be changed — integers, floats, strings, tuples, and frozensets. Immutable objects are hashable and can be used as dictionary keys or set elements; mutable objects cannot.

**Q056 What are Python modules and packages?**

A module is a single Python file (.py) containing functions, classes, and variables. A package is a directory containing multiple modules and an `__init__.py` file. Modules are imported using 'import module\_name' or 'from module import item'. Packages help organize large codebases into namespaces.

**Q057 What is the 'with' statement used for?**

The 'with' statement is used for context management — ensuring proper acquisition and release of resources. It calls `__enter__` at the start and `__exit__` at the end, even if an exception occurs. Most commonly used for file handling: 'with open("file.txt") as f:' automatically closes the file.

**Q058 What is list comprehension?**

List comprehension provides a concise way to create lists. Syntax: [expression for item in iterable if condition]. Example: `squares = [x**2 for x in range(10) if x % 2 == 0]` creates squares of even numbers. Dictionary and set comprehensions follow the same pattern using `{}` instead.

**Q059 What is a generator expression?**

A generator expression is like a list comprehension but uses parentheses: `(x**2 for x in range(10))`. Unlike list comprehensions, generator expressions produce items lazily on demand, using much less memory. Ideal for large datasets where you don't need all items at once.

**Q060 What is the difference between '/' and '//' in Python?**

'/' is true division and always returns a float (e.g., `7/2 = 3.5`). '//' is floor division and returns the largest integer less than or equal to the result (e.g., `7//2 = 3`). For negative numbers, floor division rounds toward negative infinity: `-7//2 = -4`.

**Q061 What is the difference between a set and a frozenset?**

A set is a mutable, unordered collection of unique elements defined with `{}` or `set()`. A frozenset is its immutable version created with `frozenset()`. Because frozensets are immutable, they are hashable and can be used as dictionary keys or elements of another set, unlike regular sets.

**Q062 What is the difference between 'is None' and '== None'?**

Always use 'is None' or 'is not None' rather than '== None'. Since None is a singleton in Python, 'is' checks identity and is guaranteed to work correctly. '== None' can be overridden by a class's `__eq__` method, making it unreliable. PEP 8 explicitly recommends using 'is None'.

**Q063 What is method resolution order (MRO) and how does super() use it?**

MRO is the order in which Python searches classes for a method. `super()` doesn't just call the parent class — it calls the next class in the MRO. This ensures cooperative multiple inheritance works correctly, with each class called exactly once in a predictable order defined by C3 linearization.

**Q064 What is the difference between append(), extend(), and insert() in lists?**

`append(x)` adds a single element to the end. `extend(iterable)` adds all elements of an iterable to the end. `insert(i, x)` inserts element `x` at position `i`, shifting subsequent elements. `append` and `extend` are  $O(1)$  amortized; `insert` is  $O(n)$  because it shifts elements to make room.

**Q065 What is the difference between map() and a list comprehension?**

Both apply a function to each element of an iterable. `map(func, iterable)` returns a lazy iterator and is slightly faster for simple built-in functions. List comprehensions are more readable, support filtering (if condition), and are generally preferred in Python. Use `map()` when passing an existing named function directly.

**Q066 What is \*args and \*\*kwargs in function definitions?**

\*args collects extra positional arguments into a tuple. \*\*kwargs collects extra keyword arguments into a dictionary. They allow functions to accept variable numbers of arguments. Order must be: regular args, \*args, keyword-only args, \*\*kwargs. Used widely in wrappers and decorators.

**Q067 What is the difference between 'del', 'remove()', and 'pop()' for lists?**

'del list[index]' removes an element at a specific index. 'list.remove(value)' removes the first occurrence of a specific value — raises ValueError if not found. 'list.pop(index)' removes and returns the element at the given index (default: last). pop() from the end is O(1); from middle is O(n).

**Q068 What is the difference between recursion and iteration?**

Recursion solves a problem by having a function call itself with a smaller sub-problem, using the call stack. Iteration uses loops. Recursion is elegant for tree/graph traversal but risks stack overflow for deep recursion. Iteration is generally more memory-efficient and faster for simple repetition.

**Q069 What are Python's built-in functions for sorting?**

sorted(iterable, key=None, reverse=False) returns a new sorted list without modifying the original. list.sort() sorts in-place and returns None. Both use Timsort O(n log n). The 'key' parameter accepts a function applied to each element for comparison: sorted(words, key=str.lower).

**Q070 What is the purpose of \_\_repr\_\_ vs \_\_str\_\_?**

\_\_str\_\_ returns a human-readable string for end users, used by print() and str(). \_\_repr\_\_ returns an unambiguous developer-facing representation, used by the interpreter and repr(). If only \_\_repr\_\_ is defined, it serves as fallback for \_\_str\_\_. Best practice: \_\_repr\_\_ should return a string that could recreate the object.

**Q071 What is the difference between a coroutine and a generator?**

Generators produce data for iteration using `yield`. Coroutines are designed for concurrency and can both receive and yield values using `'value = yield'`. In modern Python, `async def` coroutines (used with `asyncio`) are distinct from generator-based coroutines and are the preferred approach for async programming.

**Q072 What is the GIL and when does it matter?**

The Global Interpreter Lock (GIL) allows only one thread to execute Python bytecode at a time. It matters for CPU-bound tasks — use multiprocessing to bypass it. For I/O-bound tasks (network, file), threading is effective since threads release the GIL while waiting. Python 3.13 introduces an experimental free-threaded build.

**Q073 What is the difference between a class variable and an instance variable?**

A class variable is defined directly inside the class (outside methods) and is shared across all instances. An instance variable is defined inside `__init__` using `self.variable` and is unique to each instance. Modifying a class variable via an instance creates a new instance variable that shadows the class variable for that instance only.

**Q074 What is PEP 8 and why does it matter?**

PEP 8 is Python's official style guide. It covers naming conventions (`snake_case` for variables/functions, `PascalCase` for classes), indentation (4 spaces), max line length (79 chars), blank lines, import ordering, and commenting. Following PEP 8 ensures consistent, readable, and professionally written Python code.

**Q075 What is the difference between `type()` and `isinstance()`?**

`type(obj)` returns the exact class of an object. `isinstance(obj, class)` checks whether an object is an instance of a class or any of its subclasses. `isinstance` is preferred for type checking because it properly handles inheritance, while `type()` does not consider subclasses — making it too strict for most use cases.



**Q076 What is monkey patching in Python?**

Monkey patching is the dynamic modification of a class or module at runtime, replacing or extending its attributes or methods without changing the original source code. Useful in testing (mocking) and quick fixes, but should be used sparingly as it can make code unpredictable and difficult to maintain.

**Q077 What is the difference between `__iter__` and `__next__`?**

`__iter__` returns the iterator object itself (usually `self`) and is called when `iter()` is invoked on an object. `__next__` returns the next value and raises `StopIteration` when exhausted. A class implementing both is a full iterator. An iterable only needs `__iter__`, which returns a separate iterator object.

**Q078 What is the purpose of the 'pass' statement?**

'pass' is a no-operation placeholder used when a statement is syntactically required but no action is needed. Common uses: empty function or class bodies during development, empty except blocks when intentionally ignoring an exception, and as a placeholder in abstract method stubs before implementation.

**Q079 What is the difference between 'is' and '==' in Python?**

'==' checks for value equality — whether two objects have the same value. 'is' checks for identity — whether two variables point to the exact same object in memory. For example: `a = [1,2]; b = [1,2]; a == b` is `True` but `a is b` is `False`. Use 'is' only for `None`, `True`, and `False` checks.

**Q080 What is the difference between 'break', 'continue', and 'pass' in loops?**

'break' exits the current loop entirely, skipping any remaining iterations and the else clause. 'continue' skips the rest of the current iteration and moves to the next one. 'pass' does nothing — it is a syntactic placeholder that allows an empty loop body without causing a syntax error.

**Q081 What is a decorator in Python?**

A decorator is a function that takes another function (or class) and extends its behavior without modifying its source code. Applied with `@decorator_name` syntax above a function definition. Decorators are used for logging, timing, access control, caching, and validation, and can be stacked.

**Q082 What is a closure in Python?**

A closure is a nested function that captures and remembers variables from its enclosing scope, even after the outer function has returned. It requires a nested function referencing a variable from the enclosing function, and the enclosing function returning that nested function. Closures underpin decorators and factory functions.

**Q083 What are lambda functions?**

A lambda is an anonymous, single-expression function defined with the `lambda` keyword: `lambda x, y: x + y`. It cannot contain statements or multiple expressions, only one implicit return. Commonly used as a short callback for `sorted()`, `map()`, or `filter()` where defining a full function would be overkill.

**Q084 What does it mean that functions are 'first-class objects' in Python?**

Functions being first-class means they can be assigned to variables, stored in data structures, passed as arguments, and returned from other functions, just like any other object. This enables higher-order functions, decorators, and functional programming patterns such as passing callbacks.

**Q085 What is the LEGB rule in Python?**

LEGB describes the order Python searches for a name: Local (current function), Enclosing (any enclosing function), Global (module level), and Built-in (Python's built-in namespace). The interpreter stops at the first scope where the name is found, raising `NameError` if none match.

**Q086 Why is using a mutable default argument dangerous in Python?**

Default argument values are evaluated once, at function definition time, not on every call. If a default is a mutable object like a list or dict, that same object persists across calls and accumulates state. The fix is to default to None and create the mutable object inside the function body.

**Q087 What are positional-only parameters in Python?**

Introduced in PEP 570 (Python 3.8), a forward slash (/) in a function signature marks all parameters before it as positional-only — they cannot be passed by keyword. Example: `def f(a, b, /): ...` This lets library authors rename parameters later without breaking callers.

**Q088 What are keyword-only parameters in Python?**

An asterisk (\*) in a function signature marks all parameters after it as keyword-only — they must be passed by name, not position. Example: `def f(a, *, b): ...` requires calling `f(1, b=2)`. This improves call-site clarity for functions with many optional flags.

**Q089 What does the '=' specifier do in an f-string?**

Added in Python 3.8, writing `f'{expr=}'` prints both the expression's source text and its value, e.g. `f'{x=}'` outputs `'x=5'`. This is a quick debugging aid that avoids manually typing variable names in print statements while inspecting values.

**Q090 What is dictionary comprehension?**

Dictionary comprehension builds a dict concisely using `{key_expr: value_expr for item in iterable if condition}`. Example: `{x: x**2 for x in range(5)}` creates squares keyed by their base number. It is more readable and often faster than building a dict with an explicit loop.

**Q091 What is set comprehension?**

Set comprehension builds a set using {expression for item in iterable if condition}, automatically discarding duplicates. Example: {x % 3 for x in range(10)} produces the set of possible remainders. It follows the same syntax pattern as list and dict comprehensions but with curly braces and no colon.

**Q092 What is Python's format specification mini-language?**

The format spec mini-language controls how values are rendered inside str.format() or f-strings, using syntax like {value:width.precision type}. Examples: f'{3.14159:.2f}' gives '3.14', f'{42:05d}' gives '00042', and f'{n:,}' adds thousands separators. It also supports alignment (<, >, ^) and sign control.

**Q093 How does slicing work in Python?**

Slicing extracts a sub-sequence using sequence[start:stop:step]. start is inclusive, stop is exclusive, and step controls direction and stride. Example: 'hello'[1:4] gives 'ell', and 'hello'[::-1] reverses the string. Omitted indices default to the sequence's start or end.

**Q094 How does negative indexing work in Python?**

Negative indices count from the end of a sequence, with -1 referring to the last element, -2 the second-to-last, and so on. Example: 'python'[-1] gives 'n'. Negative indices can be combined with slicing, e.g. list[-3:] returns the last three elements.

**Q095 What does the id() function do?**

id(obj) returns a unique integer identifier for an object for the duration of its lifetime, representing its memory address in CPython. It is used with 'is' comparisons and to check whether two variables reference the same object rather than equal but distinct objects.

**Q096 What does the hash() function do, and how does it relate to \_\_eq\_\_?**

hash(obj) returns an integer hash value used by dicts and sets for fast lookups. Objects that compare equal (\_\_eq\_\_ returns True) must have the same hash value, or dict/set behavior breaks. If you override \_\_eq\_\_ on a class, you must also define a consistent \_\_hash\_\_, or the class becomes unhashable.

**Q097 What is the weakref module used for?**

The weakref module creates references to objects that don't increase their reference count, allowing the object to be garbage collected even while the weak reference exists. Useful for caches and observer patterns where you don't want to keep an object alive just because it's being tracked.

**Q098 How does garbage collection work in CPython?**

CPython primarily uses reference counting: each object tracks how many references point to it, and is deallocated immediately when the count reaches zero. A supplementary generational garbage collector (the gc module) periodically detects and cleans up reference cycles that reference counting alone cannot resolve.

**Q099 What is the gc module used for?**

The gc module exposes Python's cyclic garbage collector. Key functions: gc.collect() forces a collection cycle, gc.disable()/gc.enable() toggle automatic collection, and gc.get\_objects() lists tracked objects. It's mainly used for diagnosing memory leaks or tuning performance in long-running applications.

**Q100 What is the Enum module used for?**

The enum module lets you define a set of named, related constants as a class using Enum. Example: class Color(Enum): RED = 1; GREEN = 2. Members are compared by identity, are iterable, and print with a readable name instead of a bare value, improving code clarity over magic numbers.

### Q101 What is the difference between IntEnum, StrEnum, and IntFlag?

IntEnum members behave like plain integers and can be compared to ints directly. StrEnum (Python 3.11+) members behave like strings. IntFlag supports bitwise combination of members (e.g., Permissions.READ | Permissions.WRITE) for representing flag-like sets, while regular Enum members support none of these interoperations.

### Q102 What does the typing module provide?

The typing module supplies generic type hint constructs: List[int], Dict[str, int], Optional[str] (equivalent to Union[str, None]), and Union[int, str] for multiple possible types. Since Python 3.9+, built-in generics like list[int] are often preferred over typing.List, though typing.Optional and typing.Union are still common.

### Q103 What is a TypeVar used for in typing?

TypeVar defines a generic placeholder type so a function's input and output types stay linked. Example: T = TypeVar('T'); def first(items: list[T]) -> T. Static type checkers like mypy use this to verify that the returned value matches the type of the list's elements.

### Q104 What is a Protocol in the typing module?

Protocol (PEP 544) enables structural subtyping — a form of static duck typing. A class satisfies a Protocol if it has the required methods and attributes, without needing to explicitly inherit from it. This lets type checkers validate duck-typed code that relies on an object's shape rather than its class hierarchy.

### Q105 What does the @overload decorator do?

typing.overload lets you declare multiple type signatures for the same function name, each followed by '...' as a stub body, ending with one real implementation. Static type checkers use the overloads to pick the correct return type based on argument types, useful when behavior varies by input type.

**Q106 What are Final and Literal in the typing module?**

Final marks a variable, attribute, or method as not meant to be reassigned or overridden, and is enforced by static type checkers, not at runtime. Literal restricts a value to a specific set of literal values, e.g. `Literal['GET', 'POST']`, enabling precise validation of allowed strings or constants.

**Q107 What is a TypedDict?**

TypedDict (from typing) defines a dictionary type with a fixed set of expected string keys and value types, without creating a full class. Example: `class Point(TypedDict): x: int; y: int`. It's purely a static typing construct — at runtime, a TypedDict value is still a plain dict.

**Q108 What is the difference between typing.NamedTuple and collections.namedtuple?**

`collections.namedtuple()` is a function that creates a tuple subclass from a string of field names. `typing.NamedTuple` is a class-based syntax that additionally supports type annotations and default values, e.g. `class Point(NamedTuple): x: int; y: int = 0`. Both produce equivalent, immutable, tuple-based objects.

**Q109 What is \_\_post\_init\_\_ in a dataclass?**

`__post_init__` is a method automatically called by the generated `__init__` right after all fields are set, letting you add custom validation or compute derived fields. It's commonly used with `dataclasses.field(init=False)` to compute a value from other fields once the object is otherwise fully initialized.

**Q110 What is dataclasses.field() used for?**

`field()` customizes an individual dataclass attribute beyond a simple default value. `default_factory=list` supplies a fresh mutable default per instance (avoiding the shared-default pitfall), while other options include `repr=False`, `compare=False`, and `init=False` to exclude a field from generated methods.

**Q111 What is `functools.singledispatch` used for?**

`functools.singledispatch` turns a function into a generic function that dispatches to different implementations based on the type of its first argument. Register type-specific implementations with `@func.register(SomeType)`. It's a clean alternative to long `isinstance()` chains when behavior needs to vary by argument type.

**Q112 What is `functools.cached_property`?**

`@functools.cached_property` (Python 3.8+) computes an instance property once and caches the result in the instance's `__dict__`, so subsequent accesses skip recomputation. Unlike `@property`, the value is stored, not recalculated, making it suited for expensive derived attributes on otherwise immutable-ish objects.

**Q113 What is the `operator` module used for?**

The `operator` module exposes built-in operators as regular functions: `operator.add`, `operator.mul`, `operator.itemgetter`, `operator.attrgetter`. These are useful as key functions for `sorted()` or as callables passed to `functools.reduce()`, and are typically faster than an equivalent `lambda`.

**Q114 What does `itertools.accumulate()` do?**

`itertools.accumulate(iterable, func=operator.add)` returns an iterator of running totals (or running results of any binary function) over the input. Example:  
`list(accumulate([1,2,3,4]))` gives `[1,3,6,10]`. Passing `max` as `func` produces a running maximum instead of a running sum.

**Q115 What does `itertools.tee()` do?**

`itertools.tee(iterable, n)` splits a single iterable into `n` independent iterators that can each be consumed separately at their own pace. Because the original iterator is consumed as needed and buffered internally, once `tee()` is called, the original iterable should not be used directly.



**Q116 What does `itertools.starmap()` do?**

`itertools.starmap(func, iterable_of_tuples)` applies `func` to each tuple in the iterable by unpacking it as arguments, equivalent to `[func(*args) for args in iterable]`. It's useful when your data is already stored as tuples of arguments, avoiding a lambda wrapper just to unpack them.

**Q117 What does `itertools.compress()` do?**

`itertools.compress(data, selectors)` filters elements from `data`, keeping only those where the corresponding element in `selectors` is truthy. Example: `list(compress(['a','b','c'], [1,0,1]))` gives `['a','c']`. It's a fast way to filter one sequence using a parallel boolean mask.

**Q118 What does `itertools.filterfalse()` do?**

`itertools.filterfalse(predicate, iterable)` returns elements for which the predicate is `False` — the logical opposite of the built-in `filter()`. Example: `list(filterfalse(lambda x: x % 2 == 0, range(10)))` returns the odd numbers. It avoids writing a negated lambda inside `filter()`.

**Q119 What does `functools.total_ordering` do?**

`@functools.total_ordering` is a class decorator that fills in the remaining rich comparison methods (`__le__`, `__gt__`, `__ge__`) automatically once you define `__eq__` and just one of `__lt__` / `__le__` / `__gt__` / `__ge__`. It reduces boilerplate for classes that need full ordering support.

**Q120 What is the `unittest` module used for?**

`unittest` is Python's built-in testing framework, based on the xUnit style. Tests are written as methods on a class inheriting from `unittest.TestCase`, using assertions like `assertEqual` and `assertRaises`. Tests are grouped and run via `unittest.main()` or a test runner, with `setUp/tearDown` for fixtures.

**Q121 What is pytest and how does it differ from unittest?**

pytest is a popular third-party testing framework that allows plain functions (not classes) as test cases, uses the built-in assert statement instead of special assertion methods, and provides powerful fixtures via the `@pytest.fixture` decorator. It can also discover and run unittest-style tests, making migration easy.

**Q122 What is unittest.mock used for?**

unittest.mock provides Mock and MagicMock objects that simulate the behavior of real objects, plus the `patch()` function/decorator to temporarily replace real objects with mocks during a test. It's used to isolate the code under test from external dependencies like databases, APIs, or the filesystem.

**Q123 What is the doctest module?**

doctest scans docstrings for text that looks like an interactive Python session (with `>>>` prompts) and executes it, comparing the actual output to the expected output shown in the docstring. It keeps small usage examples in sync with the code, doubling as both documentation and a lightweight test.

**Q124 What is the timeit module used for?**

timeit measures the execution time of small code snippets accurately by running them many times and minimizing the effect of system noise. It can be used as a library (`timeit.timeit()`) or from the command line (`python -m timeit`), and is the standard way to micro-benchmark alternative implementations.

**Q125 What is cProfile used for?**

cProfile is Python's built-in deterministic profiler, reporting how much time is spent in each function call across a program's execution. Run via `python -m cProfile script.py`, it outputs call counts and cumulative time per function, helping identify real performance bottlenecks instead of guessing.

**Q126 What is code coverage and how is it measured in Python?**

Code coverage measures what percentage of your source lines (or branches) were actually executed by your test suite. The coverage.py package (often used via pytest-cov) instruments code during test runs and reports untested lines, helping identify gaps in test suites, though 100% coverage doesn't guarantee correctness.

**Q127 What is the subprocess module used for?**

subprocess lets Python spawn and interact with external processes and shell commands. subprocess.run() is the modern, recommended interface, returning a CompletedProcess with stdout, stderr, and return code. It replaces older, less safe alternatives like os.system() and os.popen().

**Q128 What is the shutil module used for?**

shutil provides high-level file operations beyond what os offers: shutil.copy() and shutil.copytree() copy files and directory trees, shutil.move() relocates files, shutil.rmtree() recursively deletes a directory, and shutil.disk\_usage() reports disk space statistics.

**Q129 What is the pathlib module?**

pathlib (Python 3.4+) provides an object-oriented way to work with filesystem paths via the Path class, replacing much of os.path's string-based API. Example: Path('data') / 'file.txt' joins paths with '/', and methods like .exists(), .read\_text(), and .glob() make common file operations more readable.

**Q130 What is the glob module used for?**

glob finds filenames matching a Unix shell-style wildcard pattern, such as glob.glob('\*.\*txt') for all text files in the current directory. It supports '\*' (any characters), '?' (single character), and with recursive=True and '\*\*', can search directories recursively.

**Q131 What is the tempfile module used for?**

tempfile creates temporary files and directories that are automatically cleaned up. `tempfile.NamedTemporaryFile()` creates a file with a real filesystem name, while `TemporaryDirectory()` creates a self-deleting directory as a context manager, both useful for scratch data that shouldn't persist after a script finishes.

**Q132 What is the csv module used for?**

The csv module reads and writes CSV (comma-separated values) files while correctly handling quoting, embedded commas, and different delimiters. `csv.reader()/csv.writer()` work with rows as lists, while `csv.DictReader()/csv.DictWriter()` work with rows as dictionaries keyed by column header.

**Q133 What is the sqlite3 module used for?**

sqlite3 is a built-in module providing a lightweight, file-based SQL database with no separate server process required. `sqlite3.connect('file.db')` opens a connection, and `cursor.execute()` runs SQL statements. It's ideal for local storage, prototyping, and small applications that don't need a full database server.

**Q134 What is the hashlib module used for?**

hashlib provides cryptographic hash functions like SHA-256 and MD5 for generating fixed-size digests of data, commonly used for checksums, password hashing (with proper salting), and data integrity verification. Example:  
`hashlib.sha256(data).hexdigest()` returns a hex string digest of the input bytes.

**Q135 What is the difference between the random and secrets modules?**

The random module generates pseudo-random numbers using a fast, deterministic algorithm suitable for simulations and games, but it is not cryptographically secure and can be predicted. The secrets module (Python 3.6+) generates cryptographically strong random values, and should always be used for tokens, passwords, and security-sensitive randomness.

**Q136 What is the uuid module used for?**

uuid generates universally unique identifiers. `uuid.uuid4()` creates a random 128-bit UUID with a negligible chance of collision, while `uuid.uuid1()` incorporates the host's MAC address and current time. UUIDs are commonly used as database primary keys or unique object identifiers across distributed systems.

**Q137 What is the datetime module used for?**

`datetime` provides classes for manipulating dates and times: `date`, `time`, `datetime`, and `timedelta`. `datetime.now()` gets the current local time, `strftime()/strptime()` format and parse date strings, and `timedelta` objects support arithmetic like adding or subtracting durations from a `datetime`.

**Q138 What is the difference between the time and datetime modules?**

The `time` module works closer to the OS level with Unix timestamps (`time.time()`) and simple sleep/measurement functions (`time.sleep()`, `time.perf_counter()`). The `datetime` module offers richer, higher-level date and time objects with calendar-aware arithmetic, formatting, and time zone support, making it preferred for most application-level date logic.

**Q139 What is the zoneinfo module?**

`zoneinfo` (Python 3.9+) provides IANA time zone support built into the standard library, replacing the need for the third-party `pytz` package in most cases. Example: `datetime.now(ZoneInfo('Asia/Kolkata'))` attaches a specific time zone to a `datetime` object, correctly accounting for daylight saving rules.

**Q140 What is the argparse module used for?**

`argparse` builds command-line interfaces by declaring expected arguments and options. `parser.add_argument()` defines a flag or positional argument, and `parser.parse_args()` parses `sys.argv` into a namespace object. It automatically generates `--help` text and handles type conversion and error messages for invalid input.

**Q141 What is the configparser module used for?**

configparser reads and writes INI-style configuration files with [section] headers and key = value pairs. config.read('file.ini') loads a file, and config['section']['key'] retrieves a value. It's commonly used for application settings that should be editable without touching source code.

**Q142 What is the difference between venv, virtualenv, and conda?**

venv is Python's built-in, lightweight tool for creating isolated environments using the same interpreter version as the system Python. virtualenv is a more feature-rich third-party predecessor to venv that also supports older Python versions. conda is a separate package and environment manager that can install non-Python dependencies and different Python versions entirely.

**Q143 What is a requirements.txt file used for?**

requirements.txt lists a project's Python package dependencies, typically with pinned versions (e.g., requests==2.31.0), so they can be installed reproducibly with pip install -r requirements.txt. It's a simple, widely supported way to share and lock down a project's dependency set.

**Q144 What is pyproject.toml used for?**

pyproject.toml (PEP 518/621) is the modern, standardized configuration file for Python projects, specifying build system requirements and project metadata (name, version, dependencies) in one place. It's replacing the older setup.py/setup.cfg approach and is used by tools like Poetry, Hatch, and pip itself.

**Q145 What is the difference between a wheel and an sdist?**

An sdist (source distribution) packages a project's raw source code, requiring a build step on install. A wheel (.whl) is a pre-built, ready-to-install binary distribution format that installs faster and doesn't require a compiler on the user's machine, making it the preferred distribution format for PyPI packages.

**Q146 What is PYTHONPATH and how does Python find modules?**

PYTHONPATH is an environment variable listing extra directories Python searches when importing modules, in addition to the standard library and installed site-packages. Python's module search order is: built-in modules, then directories in sys.path, which is initialized from the script's directory, PYTHONPATH, and installation-dependent defaults.

**Q147 What does 'if \_\_name\_\_ == "\_\_main\_\_":' do?**

When a Python file is run directly, its `__name__` variable is set to `'__main__'`. When the same file is imported as a module elsewhere, `__name__` is set to the module's name instead. This guard lets a file define reusable functions/classes while only running certain code (like a demo or CLI) when executed directly.

**Q148 What is the difference between absolute and relative imports?**

An absolute import specifies the full path from the project's root package, e.g. from `mypackage.utils` import `helper`. A relative import uses dots to reference modules relative to the current module's location, e.g. from `.utils` import `helper` (same package) or from `..utils` import `helper` (parent package). Relative imports only work inside a proper package.

**Q149 What causes a circular import in Python, and how do you fix it?**

A circular import happens when module A imports module B, and module B (directly or indirectly) imports module A back, causing an incomplete module to be referenced during initialization. Fixes include restructuring code to remove the mutual dependency, moving the import inside a function (deferred import), or merging the modules.

**Q150 What does the `__all__` variable do in a module?**

`__all__` is a list of strings defined in a module that specifies which names are exported when a client does `'from module import *'`. Names not listed in `__all__` are still accessible via explicit import but are excluded from wildcard imports, signaling the module's intended public API.

**Q151 What are namespace packages in Python?**

Namespace packages (PEP 420) allow a package's contents to be split across multiple directories without requiring an `__init__.py` file in each one. Python automatically treats a directory as a namespace package if no `__init__.py` is present, which is useful for large projects or plugin systems distributed across multiple locations.

**Q152 What is the importlib module used for?**

`importlib` provides a programmatic interface to Python's import system.

`importlib.import_module('name')` imports a module dynamically using a string name, useful when the module to load isn't known until runtime. `importlib.reload(module)` re-executes a previously imported module, useful in interactive development.

**Q153 How does Python's sys.modules cache affect imports?**

`sys.modules` is a dictionary caching every module that has already been imported in the current process, keyed by module name. Subsequent 'import' statements for the same module reuse the cached object instead of re-executing the module's code, which is why `importlib.reload()` is needed to force re-execution.

**Q154 What is the contextvars module used for?**

`contextvars` provides Context Variables that hold values which are local to the current execution context, correctly isolated across concurrently running `asyncio` tasks or threads. Unlike a plain global variable, a `ContextVar` set inside one `async` task won't leak its value into a differently-scheduled concurrent task.

**Q155 What are asyncio.Lock and asyncio.Semaphore used for?**

`asyncio.Lock` ensures only one coroutine at a time can enter a critical section, acquired and released with 'async with lock:'. `asyncio.Semaphore` generalizes this to allow up to N concurrent coroutines into a section, commonly used to cap concurrent network requests or resource usage in `async` code.



**Q156 What is `asyncio.Queue` used for?**

`asyncio.Queue` is an async-aware FIFO queue used to pass data between producer and consumer coroutines safely. `put()` and `get()` are coroutines that can be awaited, allowing consumers to block (without blocking the event loop) until an item becomes available, implementing the producer-consumer pattern asynchronously.

**Q157 What is an async generator in Python?**

An async generator is defined with `'async def'` containing a `'yield'` statement, combining asynchronous operations with lazy iteration. It's consumed with `'async for'` instead of a regular `for` loop, and is useful for streaming data (e.g., paginated API results) where each item may require an awaited I/O call to produce.

**Q158 What is an async context manager?**

An async context manager implements `__aenter__` and `__aexit__` as coroutines instead of `__enter__`/`__exit__`, and is used with `'async with'`. It's used when acquiring or releasing a resource involves an async operation, such as opening a database connection pool or an async HTTP session.

**Q159 What does `'async for'` do, and how does it differ from a regular `for` loop?**

`'async for'` iterates over an async iterator, one that implements `__aiter__` and an async `__anext__` method, awaiting each next value instead of retrieving it synchronously. It's used with async generators and other asynchronous data sources so the event loop isn't blocked while waiting for the next item.

**Q160 What is the `concurrent.futures` module used for?**

`concurrent.futures` provides a high-level interface for running callables asynchronously using a pool of workers, via `ThreadPoolExecutor` or `ProcessPoolExecutor`. `executor.submit()` returns a `Future` object representing the eventual result, and `executor.map()` applies a function across an iterable concurrently, abstracting away manual thread/process management.

### Q161 What is the difference between `ThreadPoolExecutor` and `ProcessPoolExecutor`?

`ThreadPoolExecutor` runs tasks in a pool of threads within the same process, sharing memory but still subject to the GIL — best for I/O-bound work. `ProcessPoolExecutor` runs tasks in separate processes with independent memory, bypassing the GIL for true parallelism — best for CPU-bound work, at the cost of inter-process communication overhead.

### Q162 What is `multiprocessing.Manager` used for?

`multiprocessing.Manager()` creates a manager process that hosts shared Python objects (like dict, list, or custom proxies) accessible by multiple worker processes. Because separate processes don't share memory by default, a Manager provides a coordinated way to share and mutate state across process boundaries safely.

### Q163 What limitation does pickling place on multiprocessing?

Data passed between processes in multiprocessing (arguments, return values, and objects sent via Queue/Pipe) must be picklable, since processes communicate by serializing objects. Lambdas, local functions, and objects holding unpicklable resources (like open file handles or database connections) generally can't be passed directly to worker processes.

### Q164 What does `os.fork()` do?

`os.fork()` (Unix-only) creates a new child process that is a near-exact copy of the calling process, both continuing execution from the point of the `fork()` call. The return value distinguishes the two: 0 in the child, and the child's PID in the parent. It's a low-level primitive; multiprocessing is generally preferred for portability.

### Q165 What is structural pattern matching (`match/case`) in Python?

Introduced in Python 3.10 (PEP 634), 'match subject: case pattern:' provides structural pattern matching similar to switch statements in other languages, but far more powerful — it can match sequence structure, mapping keys, class attributes, and bind variables from the matched shape, with a wildcard '\_' as a catch-all.

**Q166 What are exception groups and 'except\*' in Python?**

Python 3.11 introduced `ExceptionGroup`, which bundles multiple unrelated exceptions together (common when multiple concurrent tasks fail), and the `'except*'` syntax to selectively handle specific exception types within that group without accidentally swallowing the others. This is especially relevant for `asyncio.TaskGroup`, which raises grouped exceptions from failed tasks.

**Q167 What is the tomllib module?**

`tomllib` (Python 3.11+) is a built-in module for parsing TOML files (read-only), used heavily for configuration such as `pyproject.toml`. `tomllib.load(file)` reads a binary-mode file object and returns a dict of the parsed data; writing TOML still requires a third-party library.

**Q168 What is a custom exception class and why define one?**

A custom exception is a class that inherits from `Exception` (or a more specific built-in exception), letting you represent domain-specific error conditions, e.g. `class InsufficientFundsError(Exception): pass`. This allows calling code to catch precisely the errors it expects using `'except InsufficientFundsError'`, rather than overly broad or generic exception types.

**Q169 What is the full try/except/else/finally flow in Python?**

The `try` block holds code that might raise an exception. `except` catches and handles matching exceptions. `else` runs only if no exception occurred in `try`. `finally` always runs, whether or not an exception occurred, and is used for cleanup like closing files or releasing locks, even if the function returns early.

**Q170 How should multiple except clauses be ordered?**

`Except` clauses are checked top to bottom, and the first matching one runs — so more specific exception types must be listed before more general ones (e.g., `except ValueError` before `except Exception`), or the specific handler will never be reached because the general one catches it first.

**Q171 How do you write a custom context manager without a class?**

Using `@contextlib.contextmanager`, you write a generator function where code before 'yield' runs as `__enter__` and code after 'yield' (typically in a try/finally) runs as `__exit__`. Example: yield the resource, then clean it up after the yield statement, letting the function be used directly in a 'with' block.

**Q172 What is the difference between `contextlib.suppress()` and a try/except pass?**

`contextlib.suppress(SomeError)` is a context manager that silently ignores a specified exception type if raised inside the 'with' block, functioning like a one-line equivalent of `try/except SomeError: pass`. It's more concise and explicitly communicates intent, but should still be scoped narrowly to avoid hiding unrelated bugs.

**Q173 What are the common file open modes in Python?**

'r' opens for reading (default), 'w' truncates and opens for writing, 'a' opens for appending, and 'x' opens for exclusive creation (fails if the file exists). Appending 'b' (e.g. 'rb') opens in binary mode, and '+' (e.g. 'r+') allows both reading and writing on the same file object.

**Q174 What is buffering in Python file I/O?**

Buffering batches small reads/writes into larger chunks to reduce the number of costly system calls. `open(file, buffering=...)` lets you control this: 0 disables buffering (binary mode only), 1 enables line buffering (text mode), and any larger integer sets a fixed buffer size in bytes; the default picks a sensible size automatically.

**Q175 What is the difference between `encode()` and `decode()` for strings?**

`encode()` converts a Python str (Unicode text) into bytes using a specified encoding like UTF-8: `'café'.encode('utf-8')`. `decode()` does the reverse, converting bytes back into a str: `b'caf\xc3\xa9'.decode('utf-8')`. Mismatched encodings between encode and decode cause `UnicodeDecodeError` or garbled text.

### **Q176 What is Unicode normalization and why does it matter in Python?**

Some characters can be represented multiple equivalent ways in Unicode, e.g. an accented letter as one composed code point or as a base letter plus a combining accent. `unicodedata.normalize()` converts strings to a consistent form (NFC, NFD, NFKC, or NFKD) so that visually identical strings compare equal with `'=='`.

### **Q177 What does the `ascii()` function do?**

`ascii(obj)` returns a string representation of an object, like `repr()`, but escapes any non-ASCII characters using `\x`, `\u`, or `\U` escapes instead of printing them directly. It's useful for producing output that's guaranteed to be safely displayable in environments that don't support full Unicode.

### **Q178 What is operator precedence in Python and why does it matter?**

Operator precedence determines the order operations are evaluated in an expression without explicit parentheses, e.g. multiplication binds tighter than addition, so `2 + 3 * 4` evaluates to 14, not 20. Python follows a documented precedence table; when in doubt, adding parentheses makes evaluation order explicit and readable.

### **Q179 What is a ternary (conditional) expression in Python?**

Python's conditional expression has the form `value_if_true if condition else value_if_false`, evaluated inline. Example: `status = 'even' if n % 2 == 0 else 'odd'`. It's a concise alternative to a full `if/else` block when assigning a value based on a condition.

### **Q180 What is extended iterable unpacking in Python?**

PEP 3132 allows a starred name to capture multiple elements during unpacking: `first, *middle, last = [1,2,3,4,5]` assigns `first=1`, `last=5`, and `middle=[2,3,4]` as a list. This is useful for splitting off the head and/or tail of a sequence without slicing manually.

**Q181 What is chained assignment in Python?**

Chained assignment, `a = b = c`, assigns the same value to multiple variables in one statement, evaluating the right-hand side once and binding all names to that same object. For mutable objects, this means all variables reference the identical object, so mutating one is visible through the others.

**Q182 How does Python support returning multiple values from a function?**

Python functions can only return a single object, but that object is typically a tuple, which can then be unpacked at the call site: `def divmod2(a, b): return a // b, a % b`, then `q, r = divmod2(7, 2)`. The parentheses around a tuple are optional when returning multiple comma-separated values.

**Q183 What is the difference between a docstring and a regular comment?**

A docstring is a string literal placed as the first statement in a module, function, class, or method, stored in the object's `__doc__` attribute and retrievable at runtime via `help()` or introspection tools. A regular `'#'` comment is discarded by the parser and has no runtime representation. PEP 257 defines docstring conventions.

**Q184 What do the `help()` and `dir()` functions do?**

`help(obj)` displays an object's docstring and signature information in an interactive, readable format. `dir(obj)` returns a list of the names (attributes and methods) available on an object, useful for exploring an unfamiliar object interactively without consulting external documentation.

**Q185 What do the `vars()` function and `__dict__` attribute show?**

Most Python objects store their instance attributes in a dictionary accessible via `obj.__dict__`. `vars(obj)` is a convenient shortcut that returns this same dictionary. Called with no arguments inside a function, `vars()` returns the local namespace, equivalent to `locals()`.

**Q186 What is name mangling in Python?**

When an attribute name inside a class body starts with two leading underscores (and at most one trailing underscore), e.g. `__value`, Python automatically rewrites it internally to `__ClassName__value`. This makes accidental overriding by subclasses less likely, though the attribute remains accessible if you know the mangled name.

**Q187 What is the convention for private vs protected attributes in Python?**

Python has no true access control keywords; conventions signal intent instead. A single leading underscore (`_value`) signals 'internal use, not part of the public API' by convention only. A double leading underscore (`__value`) triggers name mangling, making accidental access from subclasses harder, though not truly private.

**Q188 What is an abstract property in Python?**

Combining `@property` with `@abstractmethod` (from the `abc` module) declares that subclasses must provide a concrete implementation of a particular property, not just a regular method. This enforces that any concrete subclass exposes a specific attribute-like interface, while the abstract base class itself cannot be instantiated.

**Q189 What is a mixin class in Python?**

A mixin is a class designed to be combined with others via multiple inheritance to add a specific piece of reusable behavior, without being meant to stand alone or represent a complete concept on its own. Example: a `JSONSerializableMixin` adding a `to_json()` method to any class that includes it.

**Q190 What is the difference between composition and inheritance?**

Inheritance models an 'is-a' relationship, where a subclass extends and specializes a parent class. Composition models a 'has-a' relationship, where an object contains and delegates to other objects to achieve behavior. Composition is often preferred for flexibility, since it avoids tight coupling to a rigid class hierarchy.



**Q191 When would you use a Protocol instead of an ABC?**

An ABC (`abc.ABC`) requires explicit inheritance and is best when you control the class hierarchy and want to enforce a contract via subclassing. A Protocol enables structural typing — any object with matching methods satisfies it without inheriting anything — useful for typing third-party or unrelated classes that already have the right shape. `@runtime_checkable` enables `isinstance()` checks against a Protocol.

**Q192 What are ParamSpec and Concatenate used for in typing?**

ParamSpec (PEP 612) captures an entire parameter signature (both positional and keyword parameters) as a single type variable, primarily to correctly type generic decorators that preserve the wrapped function's signature. Concatenate lets you prepend additional parameters to a captured ParamSpec, useful for decorators that inject an extra argument.

**Q193 How does the GIL interact with asyncio's concurrency model?**

asyncio achieves concurrency through a single-threaded event loop that cooperatively switches between coroutines at 'await' points, so the GIL is largely irrelevant to asyncio's own execution model — there's only ever one thread running Python bytecode at a time regardless. This makes asyncio well-suited for I/O-bound concurrency without needing to bypass the GIL at all.

**Q194 How do you cancel a running asyncio task?**

Calling `task.cancel()` requests cancellation by raising `asyncio.CancelledError` inside the coroutine at its next suspension point (an 'await'). The coroutine can catch `CancelledError` to perform cleanup, but should generally re-raise it, since swallowing it silently can leave the task's cancellation state ambiguous to the caller.

**Q195 What is the difference between `asyncio.wait()` and `asyncio.gather()`?**

`asyncio.gather(*coros)` runs coroutines concurrently and returns their results in order once all complete, propagating the first exception raised (with an option to return exceptions instead). `asyncio.wait(tasks)` instead returns two sets — done and pending tasks — offering more control over partial completion, timeouts, and cancellation, but does not directly return ordered results.



**Q196 How do you set a timeout on an asyncio operation?**

`asyncio.wait_for(coro, timeout=seconds)` wraps a coroutine and raises `asyncio.TimeoutError` if it doesn't complete within the given time, cancelling the underlying task. Python 3.11 introduced `'async with asyncio.timeout(seconds):'` as a more composable context-manager-based alternative for applying timeouts to a block of async code.

**Q197 What is `threading.local` used for?**

`threading.local()` creates an object whose attributes are stored separately per thread, even though all threads reference the same object. Setting `local_data.value` in one thread doesn't affect the value seen in another thread, making it useful for per-thread state like database connections without manual dictionary-based lookups keyed by thread ID.

**Q198 What does `slots=True` do on a Python dataclass?**

Passing `slots=True` to `@dataclass` (Python 3.10+) automatically generates a `__slots__` definition matching the dataclass's fields, removing the per-instance `__dict__` and reducing memory usage, the same benefit as manually defining `__slots__` on a regular class, without having to write it by hand.

**Q199 What is Python's free-threaded (no-GIL) build?**

Introduced experimentally under PEP 703 starting with Python 3.13, the free-threaded build removes the Global Interpreter Lock, allowing true multi-threaded parallel execution of Python bytecode. It requires a special build flag (or `python3.13t`), may have single-threaded performance overhead, and third-party C extensions need updates to be compatible with it.

**Q200 What is the difference between a coroutine object and an asyncio Task?**

Calling an `'async def'` function just creates a coroutine object — it does nothing until awaited or scheduled. `asyncio.create_task(coro)` wraps that coroutine in a `Task`, scheduling it to start running concurrently on the event loop right away, rather than waiting for an explicit `'await'` to begin its execution.